

The Instrument — A Plain-Language Primer

CE903 Project 3 · Managing Network for LLM Training · Coordinator briefing for the Sprint 4 demo

1. The problem, in one breath

Training a large language model means thousands of GPUs working on one job at once. They cannot work in isolation — every training step, they must stop and *synchronise their results* with one another over a network. That synchronisation traffic is huge, sudden, and happens all at the same instant. Ordinary data-centre networks choke on it, and when a single GPU fails mid-job, the whole cluster can stall waiting for a participant that will never answer. Our project asks: can smarter routing reduce the time lost to congestion and failure?

2. What we built (the “instrument”)

We did *not* buy a room of GPUs. Instead we built a simulator-driven test bench that reproduces this behaviour in software, so we can run experiments cheaply and repeatably. It has three stages, in a straight line:

1. The Workload	2. The Simulator	3. The Measurements
A GPT-2 (124M) training step is captured as a Chakra trace — a recipe file (.et) listing every compute and communication action in order.	ASTRA-sim 2.0 reads the recipe and plays it out over a modelled network (a <i>Ring</i> of NPUs at 50 GB/s), tracking exactly how long each step takes.	A logging harness reads the simulator's output and computes the numbers we care about: completion time, communication time, and the cost of a failure.

Plain analogy: the trace is sheet music, ASTRA-sim is the orchestra that plays it, and the harness is the critic in the front row timing every movement. Swap the music or change the room, and we hear what happens — without hiring a real orchestra.

3. How the team maps onto the instrument

Pod / lane	Owns this part of the instrument	People
Pod A — Injection	Makes the network <i>fail on purpose</i> : a GPU “dies” mid-run so there is something to detect and measure.	Navami (lead), Tae
Pod B — Observatory	Builds the eyes: the logging that catches the stall and turns a failure into a number — wired up before the fault, never bolted on after.	Kyle (lead), Ashraf, Roshan
The Horizon	Holds the sprint together and scouts the Sprint 5 prediction work so it does not start cold. (Your lane.)	Brad — coordination

4. What the instrument has proven so far

Test	What it shows	Headline result	Status
TC-01	The pipeline closes end-to-end: a workload runs through to completion with metrics recorded correctly.	Step time $\approx$ 95.8M cycles; 83.5% compute, 16.5% comms; no errors.	PASS
TC-02	Synchronisation traffic (ring AllReduce) behaves as theory predicts across 4, 16 and 64 NPUs.	A real finding: bandwidth saturates by 16 NPUs at GPT-2 scale — more nodes stop helping.	PASS
TC-03	We can inject a worst-case GPU failure (GSP / XID 119), survive on the remaining nodes, and measure what it cost.	Recovery penalty = 3.0% of job time; stall caught at $\sim$ 1.03M cycles.	PASS
C++ hook	Stretch: the failure is wired directly inside the simulator, not faked by comparing two runs.	Fault fires at T = 75M ns; logged automatically.	STRETCH

5. The one thing to hold in your head

Everything above is apparatus. The project's actual claim — *that predictive routing beats standard routing (ECMP) when a GPU fails* — is not tested until Sprint 5 (TC-04). The 3.0% penalty we just measured is the baseline to beat. Sprint 4 built the ruler; Sprint 5 uses it to prove the thesis. That sequencing is deliberate and on-schedule, not a gap.

Held by design for later: a more realistic Alibaba-style network topology (FR-G1), and a scale check toward the 1024-GPU target (NFR-P2). Both are parked on purpose, not forgotten.

Glossary in ten seconds: JCT = total job time · AllReduce = the all-GPUs-sync step · NPU = a simulated GPU · GSP fault = the firmware crash that kills a GPU outright · ECMP = today's "dumb" routing · Chakra/.et = the workload recipe file · ASTRA-sim = the simulator.